

FATEC DEP. ARY FOSSEN

Ciência de Dados 4º Semestre
Disciplina: Bancos e Armazéns de Dados

Relatório Técnico — Projeto Avaliativo

DATA WAREHOUSE OLIST COMPLETO

Dataset: Brazilian E-Commerce Olist

Integrantes do Grupo

Daianne Soares Silva (RA: 1141352423022)
Enrico Sablich Aoyama (RA: 1141352513022)
Wellington Bianchetti Paes (RA: 1141352513024)
Victor Lorenzo Castro Rodrigues (RA: 1141352423041)

Junho de 2026

1. Introdução

1.1 Contextualização

Este relatório documenta o projeto avaliativo da disciplina Bancos e Armazéns de Dados em Ciências de Dados, 4º semestre do curso de Ciência de Dados da Fatec Dep. Ary Fossen. O trabalho simula o ciclo completo de um projeto real de Engenharia de Dados, cobrindo desde a ingestão de dados brutos até a geração de insights visuais por meio de um Data Warehouse estruturado.

1.2 Objetivo do Projeto

O objetivo é construir um Data Warehouse (DW) completo sobre o dataset de e-commerce brasileiro da Olist, utilizando a metodologia de modelagem dimensional de Ralph Kimball. As entregas previstas são:

- Modelagem dimensional com esquema estrela (Star Schema) e SCD Type 2
- Pipeline ETL em SQL organizado em camadas: Staging → OLTP → Modelo DW → Carga
- Consultas analíticas respondendo perguntas de negócio sobre o dataset
- Quatro visualizações em Python com a biblioteca Plotly (ou power BI)
- Documentação técnica: relatório, diagrama do modelo e dicionário de dados

1.3 Sobre o Dataset Olist

O dataset utilizado é o Brazilian E-Commerce Public Dataset by Olist, disponível publicamente no Kaggle. Contém informações reais de pedidos realizados entre 2016 e 2018 na plataforma de e-commerce Olist, anonimizadas conforme requisitos de privacidade. O conjunto é composto por 9 arquivos CSV interconectados, totalizando aproximadamente 45 MB e 99.441 pedidos únicos.

Arquivo CSV	Conteúdo
olist_orders_dataset.csv	Pedidos: status, datas de compra, aprovação e entrega
olist_order_items_dataset.csv	Itens do pedido: produto, vendedor, preço e frete
olist_customers_dataset.csv	Clientes: cidade, estado, CEP
olist_products_dataset.csv	Produtos: categoria, peso, dimensões físicas
olist_sellers_dataset.csv	Vendedores: cidade, estado, CEP
olist_order_payments_dataset.csv	Formas e valores de pagamento
olist_order_reviews_dataset.csv	Avaliações: nota e comentário do comprador
olist_geolocation_dataset.csv	Coordenadas geográficas por CEP
product_category_name_translation.csv	Tradução de categorias para o inglês

2. Modelo Dimensional

2.1 Esquema Estrela (*Star Schema*)

O DW foi projetado com o esquema estrela: a tabela fato (*fact_sales*) centraliza as métricas de negócio e referencia as quatro dimensões por meio de chaves substitutas (*surrogate keys*). A escolha do esquema estrela se justifica pela simplicidade das consultas, pois os *JOINS* são sempre diretos entre a fato e as dimensões, o que resulta em melhor desempenho analítico comparado ao esquema *snow flake*.

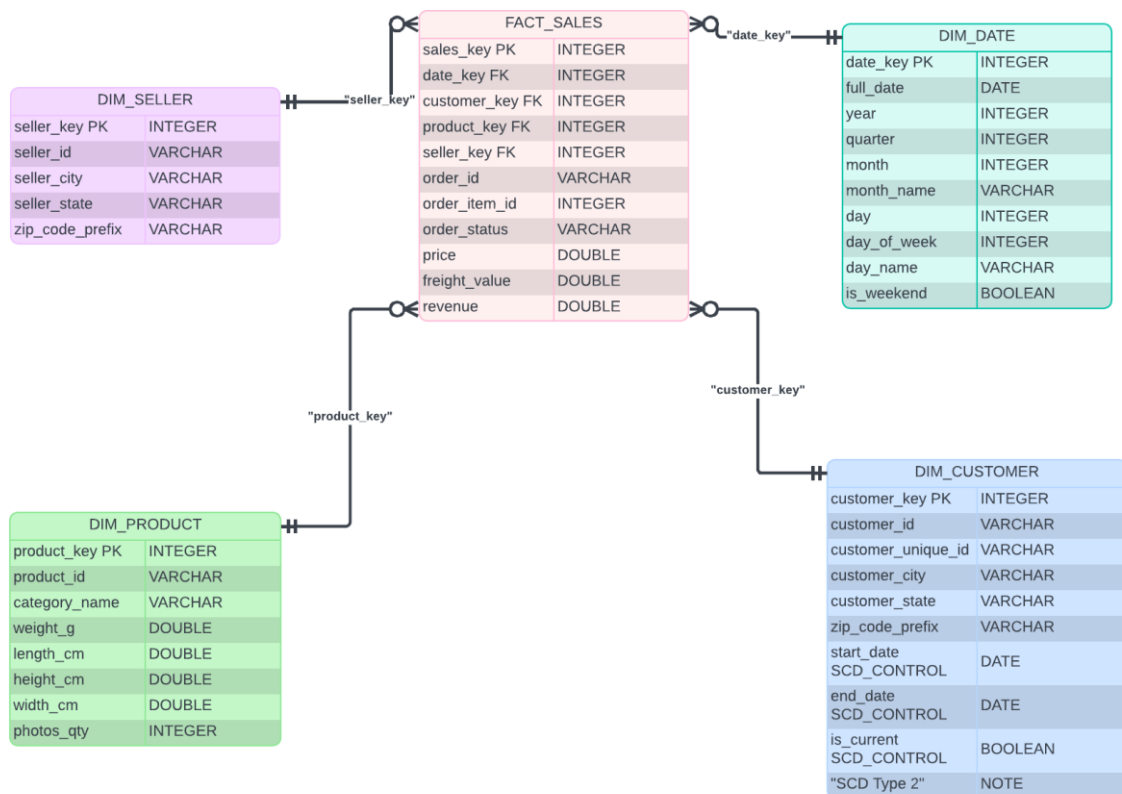


Figura 1 — Esquema Estrela do Data Warehouse Olist

2.2 Granularidade (*Grain*)

A granularidade da tabela fato foi definida ao nível de item de pedido: cada linha de *fact_sales* representa um produto vendido dentro de um pedido. Essa granularidade possibilita análises detalhadas por produto, vendedor, cliente e data, além de permitir agregações em níveis superiores (pedido, dia, mês, categoria, estado).

2.3 Chaves Substitutas

Todas as dimensões e a tabela fato utilizam chaves substitutas (*surrogate keys*) geradas por sequências DuckDB (*seq_customer*, *seq_product*, *seq_seller*, *seq_sales*), independentes dos IDs do sistema fonte. As vantagens são:

- Isolamento do DW em relação a mudanças no sistema transacional
- Suporte nativo ao *SCD Type 2*: múltiplas versões do mesmo cliente com chaves distintas

- Melhor desempenho em *JOINS* por utilizar inteiros em vez de strings UUID

2.4 Visão Geral das Tabelas

Tabela	Tipo	Descrição
dim_date	Dimensão	Calendário completo (2016–2018). Ano, mês, trimestre, dia da semana.
dim_customer	Dimensão (SCD2)	Clientes com histórico de mudanças de localização via SCD Type 2.
dim_product	Dimensão	Catálogo de produtos com categoria em inglês, peso e dimensões.
dim_seller	Dimensão	Vendedores com localização (cidade, estado, CEP).
fact_sales	Fato	Itens de pedido: métricas de preço, frete e receita total (R\$).

3. SCD Type 2 — Dimensão de Clientes

A dimensão *dim_customer* implementa o padrão *Slowly Changing Dimension Type 2 (SCD Type 2)* para rastreamento de mudanças históricas nos dados dos clientes. No contexto do Olist, a localização do cliente (cidade/estado) pode variar entre compras diferentes, e é fundamental que o DW associe cada venda ao endereço correto no momento da transação.

3.1 Funcionamento do SCD Type 2

Quando um cliente altera sua cidade ou estado, o registro anterior não é sobrescrito. Em vez disso:

- O registro antigo é encerrado: *end_date* recebe a data da mudança, *is_current* passa a *FALSE*
- Um novo registro é criado com os dados atualizados: *start_date* = data da mudança, *end_date* = *NULL*, *is_current* = *TRUE*

O exemplo abaixo ilustra um cliente que mudou de São Paulo para Campinas:

customer_key	customer_id	customer_city	start_date	end_date	is_current
1	C001	São Paulo	2020-01-01	2023-05-10	FALSE
2	C001	Campinas	2023-05-11	NULL	TRUE

Com esse mecanismo, um pedido realizado em janeiro/2020 será corretamente associado à versão do cliente em São Paulo, enquanto um pedido de junho/2023 será associado à versão em Campinas, garantindo precisão histórica nas análises.

3.2 Campos de Controle SCD Type 2

- *start_date*: data de início de validade do registro
- *end_date*: data de encerramento (NULL indica que o registro está ativo)
- *is_current*: flag booleana para identificar rapidamente o registro mais recente

4. Pipeline ETL

O pipeline ETL foi organizado em scripts SQL numerados sequencialmente, garantindo clareza na ordem de execução. Todos os scripts são idempotentes e podem ser reexecutados múltiplas vezes sem duplicar dados nem gerar erros, o que facilita o desenvolvimento iterativo e a depuração.

Script	Camada	Responsabilidade
00_staging.sql	Staging	Cria views sobre os CSVs brutos. Nenhuma transformação aplicada.
01_oltp.sql	OLTP	Normaliza, deduplica e trata nulos. Converte tipos de dados.
02_dw_model.sql	Modelo DW	Cria estrutura vazia do esquema estrela. Popula dim_date.
03_etl_load.sql	ETL Load	Carrega dimensões (com SCD Type 2) e fact_sales a partir do OLTP.
04_analytics.sql	Analytics	5 consultas SQL respondendo as perguntas de negócio.
05_performance.sql	Performance	Tabelas agregadas, índices e otimizações (bônus).

4.1 Staging (00_staging.sql)

A camada de staging cria views DuckDB apontando diretamente para os CSVs via `read_csv_auto()` com `all_varchar = true`. Nenhuma transformação é aplicada nesta etapa: os dados são lidos exatamente como estão nos arquivos-fonte. O DuckDB não copia os dados ao criar as views, tornando a leitura eficiente. O uso de `CREATE OR REPLACE VIEW` garante idempotência.

4.2 Camada OLTP (01_oltp.sql)

A camada OLTP normaliza os dados brutos em tabelas relacionais, aplicando as seguintes transformações:

- Deduplicação via `ROW_NUMBER() OVER (PARTITION BY chave)`, mantendo apenas um registro por entidade
- Tratamento de nulos com `COALESCE`: valores padrão aplicados à cidade ('desconhecido'), ao estado ('XX') e ao CEP ('00000')
- Conversão de tipos com `TRY_CAST`: datas de `VARCHAR` para `TIMESTAMP`, valores financeiros para `DOUBLE`
- Integridade referencial: itens de pedido filtrados para conter apenas referências válidas a pedidos, produtos e vendedores

4.3 Modelo Dimensional (02_dw_model.sql)

Este script cria a estrutura vazia do esquema estrela com DROP + CREATE para garantir idempotência. A dimensão dim_date é populada diretamente neste script com todos os dias do período 2016-2018, utilizando generate_series() do DuckDB, gerando 1.096 registros de datas, sem necessidade de carga ETL adicional.

4.4 Carga ETL (03_etl_load.sql)

O script 03_etl_load.sql é executado dentro de uma transação SQL e organiza a carga em quatro etapas. Primeiro, uma tabela temporária de parâmetros (etl_params) registra a data de execução, que servirá como data de vigência para o SCD Type 2. Em seguida, a carga de dim_customer aplica o SCD Type 2: registros existentes com atributos alterados têm end_date preenchido e is_current definido como FALSE; novos clientes ou novas versões são inseridos com is_current = TRUE. A carga de dim_product e dim_seller segue a estratégia Type 1 (sobrescrita), atualizando atributos alterados sem preservar histórico. Por fim, a fact_sales é recarregada integralmente com DELETE seguido de INSERT, realizando os JOINS com as dimensões para resolver as chaves substitutas. Ao final, consultas de conferência validam as contagens de linhas em cada tabela e verificam que não existe mais de uma versão ativa por cliente.

O script de carga popula as dimensões e a tabela fato a partir dos dados da camada OLTP. A carga de dim_customer implementa o SCD Type 2: novos clientes são inseridos com is_current = TRUE; clientes com alteração de localização têm o registro anterior encerrado e um novo registro ativo criado. A tabela fact_sales é populada por um INSERT que realiza os JOINS com as dimensões para obter as chaves substitutas correspondentes.

5. Consultas Analíticas

Foram desenvolvidas consultas SQL cobrindo diferentes perspectivas do negócio, utilizando JOINS entre a tabela fato e as dimensões do esquema estrela, funções de agregação e análise temporal.

5.1 Query 1 — Visão geral de vendas

Pergunta de negócio: Qual é o panorama geral de vendas do e-commerce em quantidade de pedidos, itens vendidos, valores de produtos, frete e receita?

Tabelas utilizadas: fact_sales.

Granularidade: Consolidado geral da tabela fato, sem abertura por dimensão.

Métricas: Total de pedidos, total de itens vendidos, total do preço dos produtos, total de frete, receita total e ticket médio por item.

Observações: A métrica revenue representa price + freight_value, portanto inclui produto e frete no valor de receita analisado.

5.2 Query 2 — Evolução da receita

Pergunta de negócio: Como a receita, a quantidade de pedidos e a quantidade de itens evoluíram ao longo dos meses?

Tabelas utilizadas: fact_sales, dim_date.

Granularidade: Uma linha por ano e mês da data de compra do pedido.

Métricas: Total de pedidos, total de itens vendidos, receita total e ticket médio por item.

Observações: A análise utiliza a date_key da tabela fato associada à dim_date, representando a data de compra do pedido.

```

SELECT
    d.year,
    d.month,
    d.month_name,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    COUNT(*) AS total_itens,
    ROUND(SUM(f.revenue), 2) AS receita_total,
    ROUND(AVG(f.revenue), 2) AS ticket_medio
FROM fact_sales f
JOIN dim_date d
    ON f.date_key = d.date_key
GROUP BY
    d.year,
    d.month,
    d.month_name
ORDER BY
    d.year,
    d.month;

```

5.3 Query 3 — Receita por categoria de produto

Pergunta de negócio: Quais categorias de produto geraram maior receita e qual foi o desempenho de cada categoria em pedidos, itens e frete?

Tabelas utilizadas: fact_sales, dim_product.

Granularidade: Uma linha por categoria de produto.

Métricas: Total de pedidos, total de itens vendidos, total do preço dos produtos, total de frete, receita total e ticket médio por item.

Observações: A receita considera o valor do produto somado ao frete. Categorias sem descrição tratada aparecerão conforme foram carregadas na dimensão de produto.

```

SELECT
    p.product_category_name,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    COUNT(*) AS total_itens,
    ROUND(SUM(f.price), 2) AS total_produtos,
    ROUND(SUM(f.freight_value), 2) AS total_frete,
    ROUND(SUM(f.revenue), 2) AS receita_total,
    ROUND(AVG(f.revenue), 2) AS ticket_medio
FROM fact_sales f
JOIN dim_product p
    ON f.product_key = p.product_key
GROUP BY
    p.product_category_name
ORDER BY
    receita_total DESC;

```

5.4 Query 4 — Top 10 produtos por receita

Pergunta de negócio: Quais produtos individuais geraram maior receita no e-commerce?

Tabelas utilizadas: fact_sales, dim_product.

Granularidade: Uma linha por produto individual, identificado por `product_id` e sua respectiva categoria.

Métricas: Quantidade de itens vendidos, quantidade de pedidos e receita total.

Observações: O ranking é limitado aos 10 produtos com maior receita total. A receita considera produto mais frete.

```
SELECT
  p.product_id,
  p.product_category_name,
  COUNT(*) AS qtd_itens_vendidos,
  COUNT(DISTINCT f.order_id) AS qtd_pedidos,
  ROUND(SUM(f.revenue), 2) AS receita_total
FROM fact_sales f
JOIN dim_product p
  ON f.product_key = p.product_key
GROUP BY
  p.product_id,
  p.product_category_name
ORDER BY
  receita_total DESC
LIMIT 10;
```

5.5 Query 5 — Receita por estado do cliente

Pergunta de negócio: Quais estados dos clientes geraram maior receita considerando a localização histórica no momento da compra?

Tabelas utilizadas: `fact_sales`, `dim_customer`.

Granularidade: Uma linha por estado histórico do cliente.

Métricas: Total de pedidos, total de clientes únicos, receita total e ticket médio por item.

Observações: Como a `dim_customer` utiliza SCD Type 2, a análise considera o estado válido do cliente na data da compra, e não necessariamente o estado atual.

```
SELECT
  c.customer_state,
  COUNT(DISTINCT f.order_id) AS total_pedidos,
  COUNT(DISTINCT c.customer_unique_id) AS total_clientes,
  ROUND(SUM(f.revenue), 2) AS receita_total,
  ROUND(AVG(f.revenue), 2) AS ticket_medio_item
FROM fact_sales f
JOIN dim_customer c
  ON f.customer_key = c.customer_key
GROUP BY
  c.customer_state
ORDER BY
  receita_total DESC;
```

5.6 Query 6 — Receita por cidade do cliente

Pergunta de negócio: Quais cidades dos clientes geraram maior receita considerando a localização histórica no momento da compra?

Tabelas utilizadas: `fact_sales`, `dim_customer`.

Granularidade: Uma linha por estado e cidade históricos do cliente.

Métricas: Total de pedidos, total de clientes únicos e receita total.

Observações: O ranking é limitado às 20 cidades com maior receita. A cidade exibida corresponde à versão histórica do cliente associada à venda por meio do SCD Type 2.

```
SELECT
    c.customer_state,
    c.customer_city,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    COUNT(DISTINCT c.customer_unique_id) AS total_clientes,
    ROUND(SUM(f.revenue), 2) AS receita_total
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_key = c.customer_key
GROUP BY
    c.customer_state,
    c.customer_city
ORDER BY
    receita_total DESC
LIMIT 20;
```

5.7 Query 7 — Receita por vendedor

Pergunta de negócio: Quais vendedores geraram maior receita e qual foi o desempenho deles em pedidos, itens e ticket médio?

Tabelas utilizadas: fact_sales, dim_seller.

Granularidade: Uma linha por vendedor, considerando também cidade e estado do vendedor.

Métricas: Total de pedidos, total de itens vendidos, receita total e ticket médio por item.

Observações: O ranking é limitado aos 20 vendedores com maior receita total.

```
SELECT
    s.seller_id,
    s.seller_state,
    s.seller_city,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    COUNT(*) AS total_itens,
    ROUND(SUM(f.revenue), 2) AS receita_total,
    ROUND(AVG(f.revenue), 2) AS ticket_medio_item
FROM fact_sales f
JOIN dim_seller s
    ON f.seller_key = s.seller_key
GROUP BY
    s.seller_id,
    s.seller_state,
    s.seller_city
ORDER BY
    receita_total DESC
LIMIT 20;
```

5.8 Query 8 — Receita por status do pedido

Pergunta de negócio: Como a receita e o volume de vendas se distribuem entre os diferentes status dos pedidos?

Tabelas utilizadas: fact_sales.

Granularidade: Uma linha por status do pedido.

Métricas: Total de pedidos, total de itens vendidos e receita total.

Observações: A consulta permite identificar quanto da receita está associada a pedidos entregues, cancelados ou em outros status disponíveis na base.

```
SELECT
    order_status,
    COUNT(DISTINCT order_id) AS total_pedidos,
    COUNT(*) AS total_itens,
    ROUND(SUM(revenue), 2) AS receita_total
FROM fact_sales
GROUP BY
    order_status
ORDER BY
    receita_total DESC;
```

5.9 Query 9 — Comparação dias úteis x finais de semana

Pergunta de negócio: Existe diferença no desempenho de vendas entre dias úteis e finais de semana?

Tabelas utilizadas: fact_sales, dim_date.

Granularidade: Uma linha por tipo de dia: dia útil ou final de semana.

Métricas: Total de pedidos, total de itens vendidos, receita total e ticket médio por item.

Observações: A classificação de final de semana vem do campo is_weekend da dimensão de datas.

```
SELECT
    CASE
        WHEN d.is_weekend THEN 'Final de semana'
        ELSE 'Dia útil'
    END AS tipo_dia,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    COUNT(*) AS total_itens,
    ROUND(SUM(f.revenue), 2) AS receita_total,
    ROUND(AVG(f.revenue), 2) AS ticket_medio_item
FROM fact_sales f
JOIN dim_date d
    ON f.date_key = d.date_key
GROUP BY
    d.is_weekend
ORDER BY
    receita_total DESC;
```

5.10 Query 10 — Ranking mensal com variação de receita

Pergunta de negócio: Qual foi a evolução da receita mês a mês e qual foi a variação em relação ao mês anterior?

Tabelas utilizadas: fact_sales, dim_date.

Granularidade: Uma linha por ano e mês da data da compra.

Métricas: Receita total, variação absoluta da receita e variação percentual da receita em relação ao mês anterior.

Observações: A função de janela LAG compara a receita de cada mês com a receita do mês imediatamente anterior na série temporal.

```
WITH vendas_mensais AS (  
  SELECT  
    d.year,  
    d.month,  
    d.month_name,  
    SUM(f.revenue) AS receita_total  
  FROM fact_sales f  
  JOIN dim_date d  
    ON f.date_key = d.date_key  
  GROUP BY  
    d.year,  
    d.month,  
    d.month_name  
)  
  
SELECT  
  year,  
  month,  
  month_name,  
  ROUND(receita_total, 2) AS receita_total,  
  ROUND(  
    receita_total - LAG(receita_total) OVER (  
      ORDER BY year, month  
    ),  
    2  
  ) AS variacao_absoluta,  
  ROUND(  
    100.0 * (  
      receita_total - LAG(receita_total) OVER (  
        ORDER BY year, month  
      )  
    ) / NULLIF(  
      LAG(receita_total) OVER (  
        ORDER BY year, month  
      ),  
      0  
    ),  
    2  
  ) AS variacao_percentual  
FROM vendas_mensais  
ORDER BY  
  year,  
  month;
```

5.11 Query 11 — Clientes com maior receita

Pergunta de negócio: Quais clientes únicos geraram maior receita ao longo do histórico de compras?

Tabelas utilizadas: fact_sales, dim_customer.

Granularidade: Uma linha por customer_unique_id.

Métricas: Total de pedidos, receita total e ticket médio por item.

Observações: A receita é agregada por customer_unique_id para não dividir o mesmo cliente em várias linhas caso ele tenha mudado de cidade ou estado. A cidade e o estado exibidos representam a versão atual do cliente, identificada por is_current = TRUE.

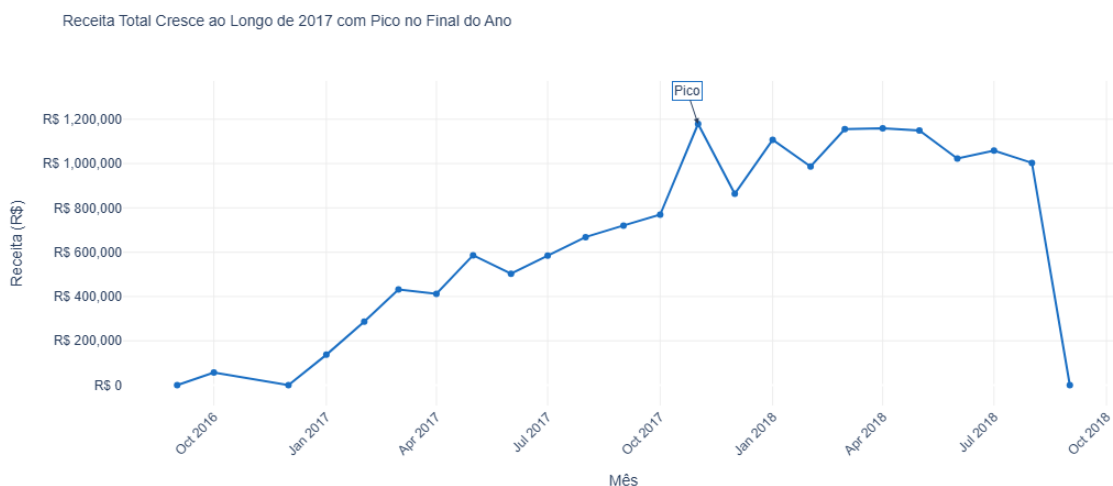
```
WITH receita_por_cliente AS (
  SELECT
    c.customer_unique_id,
    COUNT(DISTINCT f.order_id) AS total_pedidos,
    ROUND(SUM(f.revenue), 2) AS receita_total,
    ROUND(AVG(f.revenue), 2) AS ticket_medio_item
  FROM fact_sales f
  JOIN dim_customer c
    ON f.customer_key = c.customer_key
  GROUP BY
    c.customer_unique_id
)

SELECT
  rpc.customer_unique_id,
  c.customer_state AS customer_state_atual,
  c.customer_city AS customer_city_atual,
  rpc.total_pedidos,
  rpc.receita_total,
  rpc.ticket_medio_item
FROM receita_por_cliente rpc
JOIN dim_customer c
  ON c.customer_unique_id = rpc.customer_unique_id
  AND c.is_current = TRUE
ORDER BY
  rpc.receita_total DESC
LIMIT 20;
```

6. Visualizações

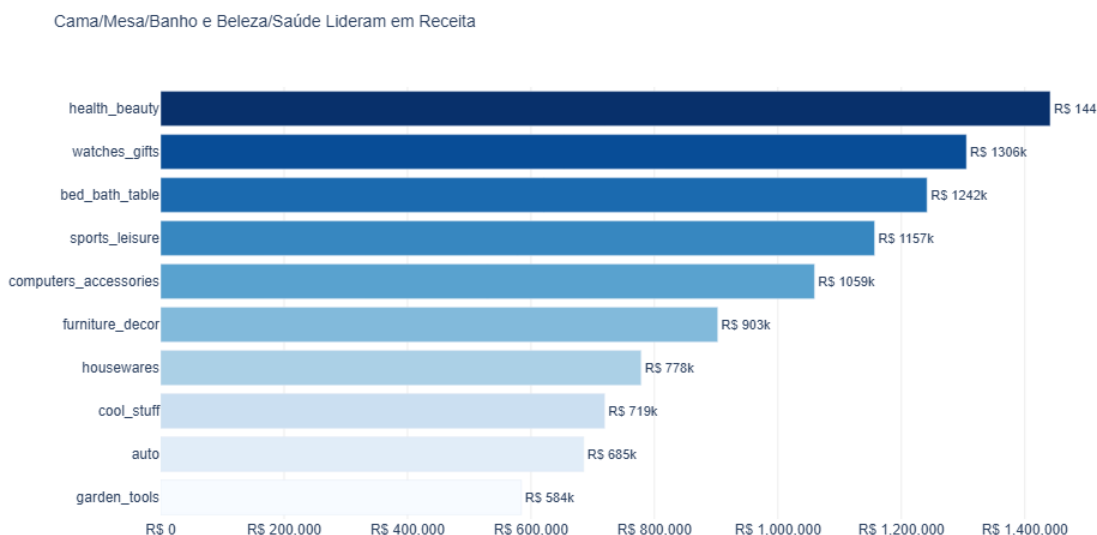
As visualizações foram geradas em Python com a biblioteca Plotly, utilizando dados extraídos diretamente do banco DuckDB por meio de consultas ao modelo dimensional. Os arquivos de saída foram salvos na pasta visualizacoes/ do repositório: três gráficos estáticos em formato PNG e um dashboard interativo em HTML.

6.1 Gráfico 1 — Evolução Mensal da Receita (Linha)



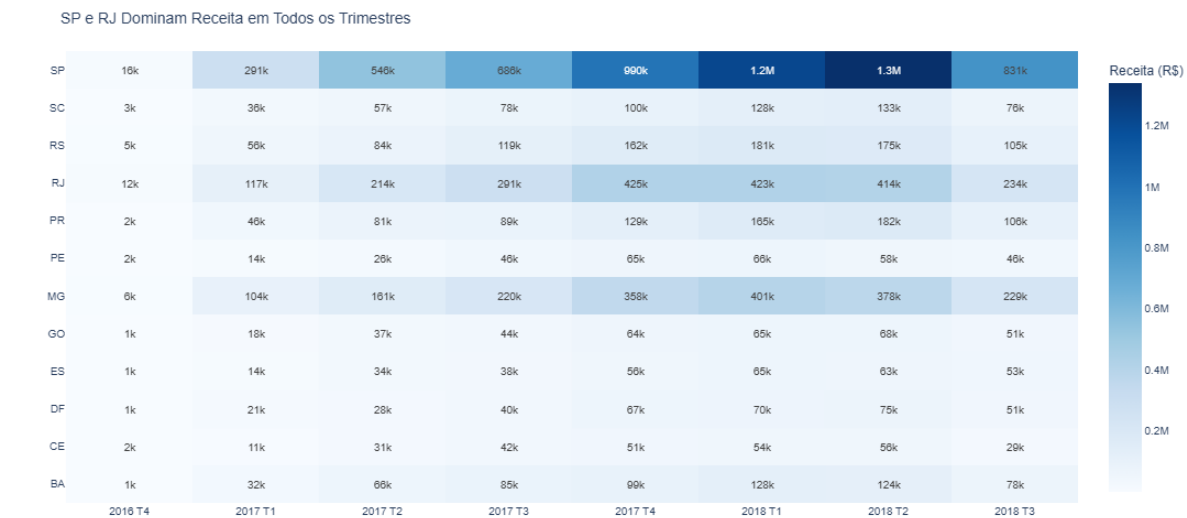
Insight: O gráfico de linha revela uma trajetória de crescimento consistente ao longo de todo o período analisado. A receita parte de valores próximos a zero em setembro de 2016, quando a plataforma ainda estava em fase inicial de operação, e atinge seu pico em novembro de 2017, mês associado à Black Friday (evento que historicamente concentra o maior volume de compras no e-commerce brasileiro). Após o pico, a receita se estabiliza em patamares elevados durante o primeiro semestre de 2018, com leve recuo nos meses finais do período, provavelmente reflexo de dados parciais nos últimos meses do dataset. O crescimento observado indica consolidação contínua da plataforma Olist ao longo dos dois anos analisados.

6.2 Gráfico 2 — Top 10 Categorias por Receita (Barras)



Insight: O gráfico de barras horizontais evidencia que as categorias `health_beauty`, `watches_gifts` e `bed_bath_table` concentram as maiores receitas, juntas respondendo por aproximadamente 25% do faturamento total da plataforma. Esse padrão sugere que produtos de beleza, acessórios de alto valor e artigos de cama/ mesa/ banho têm alta representatividade tanto em volume quanto em ticket médio no e-commerce brasileiro. A distribuição entre as dez categorias mostra uma curva relativamente suave, sem domínio absoluto de uma única categoria, o que indica diversificação saudável do portfólio de produtos na plataforma Olist.

6.3 Gráfico 3 — Receita por Estado e Trimestre (Mapa de Calor)



Insight: O mapa de calor confirma a forte concentração geográfica das vendas na região Sudeste. São Paulo domina de forma expressiva em todos os trimestres, chegando a representar mais de R\$ 1,3 milhão em um único trimestre (2018 T2). O Rio de Janeiro aparece consistentemente como segundo estado mais relevante, seguido por Minas Gerais e Paraná. Estados do Norte e Nordeste, como AM e PA, apresentam valores muito baixos ao longo de todo o período, apontando oportunidade de expansão para essas regiões. O padrão temporal revela crescimento generalizado a partir de 2017 T1, com todos os estados registrando aumento progressivo de receita até meados de 2018.

6.4 Dashboard Completo



Insight: O dashboard interativo consolida os principais indicadores de negócio em uma única interface acessível via navegador. No topo, seis cards de KPI apresentam os números mais relevantes do período: receita total, total de pedidos, ticket médio, clientes únicos, produtos distintos e número de vendedores ativos. Abaixo, três gráficos interativos permitem análise dinâmica: a linha de evolução mensal contextualiza o crescimento temporal; o gráfico de barras das categorias facilita comparações de desempenho entre segmentos de produto; e o mapa de calor por estado e trimestre revela padrões geográficos e sazonais de forma visual e intuitiva. Todos os gráficos respondem a hover com dados detalhados, tornando o dashboard uma ferramenta prática de análise exploratória.

7. Considerações Finais

[Esta seção será preenchida ao final do projeto]

O projeto permitiu vivenciar o ciclo completo de construção de um Data Warehouse, desde a ingestão de dados brutos até a geração de insights visuais. A utilização do DuckDB mostrou-se adequada para o volume de dados do Olist, oferecendo desempenho analítico satisfatório sem necessidade de infraestrutura adicional.

A implementação do SCD Type 2 na dimensão de clientes representou o maior desafio técnico, exigindo atenção à detecção correta de mudanças e ao gerenciamento de versões históricas. Por outro lado, a organização em scripts numerados e a garantia de idempotência tornaram o desenvolvimento iterativo mais seguro.

Apêndice A — Dicionário de Dados

Este apêndice descreve todas as tabelas do Data Warehouse: nome da coluna, tipo de dado, classificação de chave e descrição funcional. As abreviações de chave utilizadas são: PK (Primary Key), FK (Foreign Key) e NK (Natural Key — chave do sistema fonte).

A.1 dim_date

Dimensão de data. Cobre o período de 01/01/2016 a 31/12/2018 com granularidade diária (1.096 registros).

Coluna	Tipo	Chave	Descrição
date_key	INTEGER	PK	Chave substituta no formato YYYYMMDD (ex: 20170315)
full_date	DATE		Data completa no formato YYYY-MM-DD
year	INTEGER		Ano (ex: 2017)
quarter	INTEGER		Trimestre do ano (1 a 4)
month	INTEGER		Número do mês (1 a 12)
month_name	VARCHAR		Nome do mês em inglês (ex: January)
day	INTEGER		Dia do mês (1 a 31)
day_of_week	INTEGER		Dia da semana: 0 = domingo, 6 = sábado
day_name	VARCHAR		Nome do dia em inglês (ex: Monday)
is_weekend	BOOLEAN		TRUE se sábado ou domingo, FALSE caso contrário

A.2 dim_customer

Dimensão de clientes com SCD Type 2. Cada versão histórica de localização de um cliente ocupa uma linha distinta.

Coluna	Tipo	Chave	Descrição
customer_key	INTEGER	PK	Chave substituta gerada pela sequência seq_customer
customer_id	VARCHAR	NK	Chave de negócio do pedido no sistema fonte Olist
customer_unique_id	VARCHAR		Identificador único do cliente (agrupa todos os pedidos do mesmo comprador)
customer_city	VARCHAR		Cidade do cliente
customer_state	VARCHAR(2)		Sigla do estado brasileiro (ex: SP, RJ, MG)
customer_zip_code_prefix	VARCHAR(5)		5 primeiros dígitos do CEP do cliente
start_date	DATE		Data de início de validade deste registro — controle SCD2
end_date	DATE		Data de encerramento da validade (NULL = registro ativo) — controle SCD2
is_current	BOOLEAN		TRUE se for o registro mais recente do cliente — controle SCD2

A.3 dim_product

Catálogo de produtos do Olist com categoria em inglês e atributos físicos.

Coluna	Tipo	Chave	Descrição
product_key	INTEGER	PK	Chave substituta gerada pela sequência seq_product
product_id	VARCHAR	NK	Identificador original do produto no sistema fonte
product_category_name	VARCHAR		Categoria do produto em inglês (via tabela de tradução)
product_weight_g	DOUBLE		Peso do produto em gramas
product_length_cm	DOUBLE		Comprimento do produto em centímetros
product_height_cm	DOUBLE		Altura do produto em centímetros
product_width_cm	DOUBLE		Largura do produto em centímetros
product_photos_qty	INTEGER		Quantidade de fotos do produto no anúncio

A.4 dim_seller

Dimensão de vendedores com informações de localização.

Coluna	Tipo	Chave	Descrição
seller_key	INTEGER	PK	Chave substituta gerada pela sequência seq_seller
seller_id	VARCHAR	NK	Identificador original do vendedor no sistema fonte
seller_city	VARCHAR		Cidade onde o vendedor está localizado
seller_state	VARCHAR(2)		Sigla do estado do vendedor (ex: SP, MG)
seller_zip_code_prefix	VARCHAR(5)		5 primeiros dígitos do CEP do vendedor

A.5 fact_sales

Tabela fato com granularidade de item de pedido. Cada linha representa um produto vendido dentro de um pedido, com as métricas de preço, frete e receita total.

Coluna	Tipo	Chave	Descrição
sales_key	INTEGER	PK	Chave substituta gerada pela sequência seq_sales
date_key	INTEGER	FK → dim_date	Referência à data de compra do pedido
customer_key	INTEGER	FK → dim_customer	Referência ao cliente (versão ativa no momento da compra)
product_key	INTEGER	FK → dim_product	Referência ao produto vendido
seller_key	INTEGER	FK → dim_seller	Referência ao vendedor responsável pelo item
order_id	VARCHAR		ID do pedido original — mantido para rastreabilidade
order_item_id	INTEGER		Sequência do item dentro do pedido (1, 2, 3...)
order_status	VARCHAR		Status final do pedido (delivered, canceled, shipped...)
price	DOUBLE		Preço do produto no momento da compra (R\$)
freight_value	DOUBLE		Valor do frete pago pelo cliente (R\$)
revenue	DOUBLE		Receita total do item: price + freight_value (R\$)